SubSense — Core AI Capabilities

Overview

SubSense's AI is not a calculator. It is a **decision copilot** — one that learns who you are, remembers what you said, and intervenes at the exact moment your instinct is about to cost you money.

The AI operates across three judgment layers. Each layer targets a specific failure point in how users relate to their subscriptions. Together, they form a continuous loop: **learn → detect → intervene**.

Judgment Layer 1: Fit Assessment

> *"Does this subscription actually belong in my life?"*

The Problem

When a user considers a new subscription, the internal decision process looks like this:

> *Netflix looks great. But I already have Disney+... maybe I need both? I'll just grab the free trial and figure it out later.*

This is not irrational. It is the default human behavior under low-stakes, high-friction decisions. The user is not wrong to feel conflicted — they simply lack the full picture. They forget what they already have. They underestimate how easy it is to forget to cancel. The internal conflict resolves itself not through clarity, but through **inertia**. The subscription gets added. The question disappears.

What AI Changes

The AI does not tell users what to do. It **shows them what they can't see on their own** — a comprehensive view assembled from data they provided but never connected:

- What they already subscribe to and how actively they use each one
- What their stated preferences actually map to in terms of content overlap
- How their historical decisions played out (did past "trial" subscriptions survive?)
- Where they currently stand against their own budget

From these inputs, the AI produces a **Fit Score** (0–100) paired with a clear, specific suggestion — not a generic warning, but a personalized recommendation grounded in the user's own data.

Functional Highlights

Highlight	What It Means
Emotionally aware	The AI does not penalize a music enthusiast for having five music subscriptions. It understands context. Its judgment adapts to the individual, not a universal standard.

Highlight	What It Means
Perfect recall	The AI remembers every subscription the user has ever added, every usage pattern they reported, every evaluation they ran. Humans forget. The AI does not.
Blind spot detection	The most valuable output is not what the user already suspects — it is what they did not notice. Overlapping services, budget creep, a pattern of abandoned trials. The AI surfaces these without being asked.

Technical Highlights

Component	Role
User Profile Database	Stores subscription history, usage logs, preferences, and past evaluation outcomes. This is the foundation everything else is built on.
LLM Reasoning Engine	Takes the user profile as context and generates personalized questions, interprets free-text responses, and produces the Fit Score with reasoning. This is where personalization happens — not through rules, but through the model's ability to weigh multiple signals simultaneously.
Pre-decision Memory	Before the user even asks a question, the system has already loaded their relevant history. The LLM does not start from scratch — it starts from a fully informed baseline. This is what makes the output feel like advice from someone who knows you, rather than a generic chatbot response.

Judgment Layer 2: Anomaly Detection

> "*Something changed. Did you notice?*"

The Problem

Users are remarkably good at justifying inactivity in the moment:

> *I'm just busy this week. I'll go to the gym next week. It's fine.*

This is not dishonesty. It is a **cognitive blind spot**. When you are inside the experience, a single missed week feels like an exception. But from the outside — looking at four weeks of data — it is clearly a trend. The user cannot see the pattern because they are living inside it, one week at a time.

The result: subscriptions quietly become dead weight. By the time the user notices, months of payments have passed.

What AI Changes

The AI does not wait for the user to ask. It **watches for you**.

Every weekly check-in feeds data into a continuous tracking system. The AI distinguishes between two types of changes:

- **Occasional dips** — normal life variance. A busy week, a holiday, a schedule change. These do not trigger alerts.
- **Systematic decline** — a consistent shift in behavior over multiple weeks. This triggers a proactive intervention, before the next billing cycle.

When the AI detects an anomaly, it does not just flag it. It **proposes a plan**: pause, downgrade, or cancel — with timing suggestions tied to the user's billing schedule.

Functional Highlights

Highlight	What It Means
User-built, AI-maintained database	The data comes from the user (weekly check-ins, swipe responses, notes). But the AI is the one organizing it, cleaning it, and turning it into something the user could never build themselves — a structured, searchable history of their own behavior.
Trend intelligence	The core value is not detecting that a subscription is unused. Any app can do that. The value is **distinguishing occasional from systematic** — knowing when "I'll use it next week" is true and when it isn't. This requires pattern recognition across weeks, not a single-point check.
On-time intervention	The AI intervenes at the decision-relevant moment — before the next payment, not after. This is the difference between prevention and regret.

Technical Highlights

Component	Role
Personal Usage Database	Weekly check-in responses are stored as structured time-series data. Each entry includes usage status, emotional signal, and optional notes. This database is entirely user-sourced — no third-party data required.
Pattern Analysis Engine	The LLM receives the full usage history for each subscription and classifies the trajectory. It does not just count unused weeks — it evaluates whether the pattern is statistically meaningful or within normal variance.
Proactive Notification System	When an anomaly is confirmed, the system pushes an intervention to the user — not a generic reminder, but a specific recommendation tied to their data, their billing date, and their history with that subscription.

Judgment Layer 3: Long-term Goal Alignment

> *"You said you wanted to change. Does this actually help?"*

The Problem

Impulse purchases are not random. They follow a pattern:

> *Oh wow, this cooking class looks incredible. Gordon Ramsay! I'm definitely doing this. Let's go.*

The excitement is real. The intention is sincere. But the user has a history. They signed up for Coursera last year — used it twice. They bought Audible six months ago — listened to one book. They have a consistent pattern: **high enthusiasm, low follow-through**, specifically in the "self-improvement" category.

The user does not see this pattern. Each new purchase feels like a fresh start. The AI is the only entity in the loop that remembers all the previous fresh starts.

What AI Changes

The AI does not lecture. It does not say "you always do this." Instead, it **reframes the decision** using the user's own data:

- It acknowledges the enthusiasm (this is genuine interest, not stupidity)
- It presents the historical pattern honestly but without judgment
- It proposes an **alternative path** that preserves the motivation while reducing the financial risk — for example, a free trial with a built-in follow-up check, or a lower-commitment version of the same service

The goal is not to stop the user from subscribing. The goal is to make sure that **if they do subscribe, it actually sticks**.

Functional Highlights

Highlight	What It Means
Commitment memory	The AI remembers every goal the user has set, every intention they expressed, and every outcome that followed. When a new subscription echoes a past pattern, the AI connects the dots — not to shame, but to inform.
Behavioral prediction	Based on the user's historical trajectory with similar subscriptions (same category, same enthusiasm level, same context), the AI estimates the probability that this new subscription will follow the same path. This prediction is grounded in the user's own data, not industry averages.
Judgment without bias	The AI does not apply a moral framework to spending. It does not decide that cooking classes are frivolous or that gym memberships are virtuous. It evaluates alignment with the user's **stated goals and revealed preferences** — two things that are often in conflict, and the AI's job is to surface that conflict clearly.

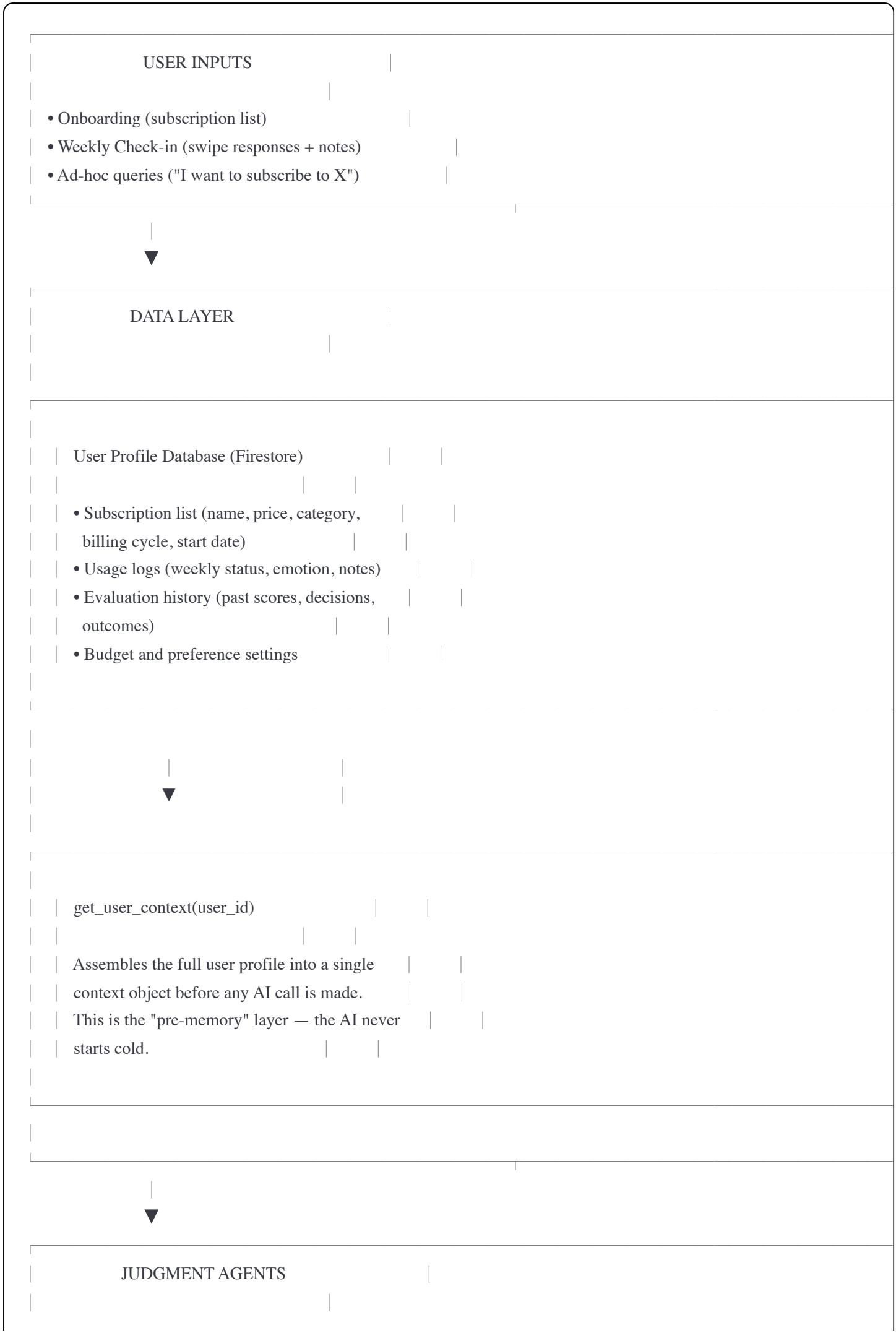
Technical Highlights

Component	Role
Longitudinal User	The AI maintains a continuously updated model of the user's behavior across all categories.

Component	Role
Model	This is not a snapshot — it is a living history that grows with every check-in and every evaluation.
Cross-category Pattern Recognition	The LLM does not analyze subscriptions in isolation. It looks across categories to identify user-level patterns: impulse tendencies, follow-through rates, category-specific enthusiasm vs. retention.
Alternative Path Generation	When the AI identifies a high-risk purchase, it does not simply say "no." It generates a concrete alternative — a lower-commitment path that tests the user's actual intent before real money is spent.

Technical Pipeline

The three judgment layers share a unified technical architecture. The pipeline below shows how data flows from user input through to AI output, and how each layer plugs into the same foundation.


USER INPUTS
• Onboarding (subscription list)
• Weekly Check-in (swipe responses + notes)
• Ad-hoc queries ("I want to subscribe to X")
DATA LAYER
User Profile Database (Firestore)
• Subscription list (name, price, category, billing cycle, start date)
• Usage logs (weekly status, emotion, notes)
• Evaluation history (past scores, decisions, outcomes)
• Budget and preference settings
get_user_context(user_id)
Assembles the full user profile into a single context object before any AI call is made.
This is the "pre-memory" layer — the AI never starts cold.
JUDGMENT AGENTS

Each layer is a separate prompt + instruction set,
but all call the same Claude API endpoint.

LAYER 1: Fit Assessment Agent

Trigger: User inputs a new subscription name

Input: user_context + new subscription info
Process: Generate personalized questions →
Collect user responses →
Analyze overlap + budget + history →
Produce Fit Score + suggestion
Output: { score, questions, reasoning,
suggestion, risk_flags }

LAYER 2: Anomaly Detection Agent

Trigger: Weekly check-in data is recorded

Input: user_context (focus on usage_logs)
Process: Classify each subscription trajectory →
Flag systematic declines →
Generate intervention if threshold met
Output: { alerts[], trend_map,
intervention_plan }

LAYER 3: Goal Alignment Agent

Trigger: Fit Score flags impulse risk

Input: user_context + new subscription info
Process: Identify cross-category behavioral

patterns → Estimate follow-through
probability → Generate alternative path
Output: { risk_level, pattern_summary,
alternative_plan, follow_up_prompt }

▼

CLAUDE API

Model: claude-sonnet-4-5-20250929

- Single endpoint for all three layers
- Each layer uses a different system prompt
- User context is injected as structured input
- Output is constrained to JSON for reliable parsing

Cost per call:
Fit Assessment: ~$0.06 (multi-turn)
Anomaly Detection: ~$0.03 (single-turn, per subscription)
Goal Alignment: ~$0.04 (single-turn, triggered)

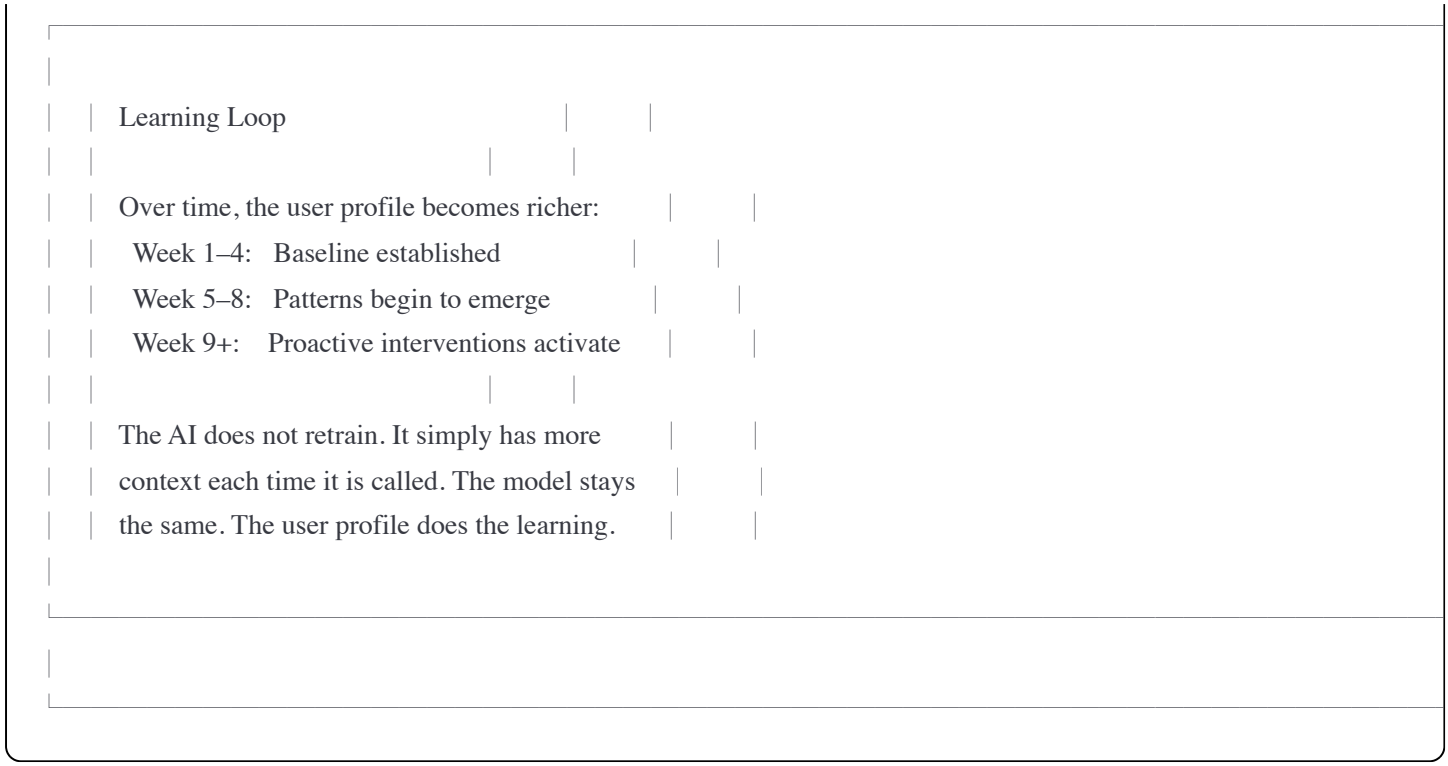
▼

OUTPUT + FEEDBACK LOOP

AI Response → User sees result

User action is recorded:
- "Subscribed anyway" → logged
- "Took the suggestion" → logged
- "Set follow-up reminder" → logged

This feedback re-enters the User Profile Database
→ Next time the AI has more to work with.



Key Architectural Decisions

Decision	Rationale
Three separate agents, one API	Each layer has a different trigger, a different prompt, and a different output schema. Keeping them separate makes each one testable and improvable independently. But they all hit the same Claude API — no need for multiple models or infrastructure.
Context-first, not rule-first	The AI does not run through a checklist of rules. It receives the user's full context and reasons over it. This is what enables personalization — a rule-based system cannot distinguish a music enthusiast from a casual listener. The LLM can.
User profile as the learning mechanism	There is no model fine-tuning or retraining in the MVP. The AI "learns" by having access to a growing database of user behavior. Each check-in makes the next AI call more informed. This keeps the architecture simple and the cost predictable.
JSON-constrained outputs	All AI outputs are constrained to structured JSON. This eliminates parsing ambiguity, makes the frontend rendering predictable, and dramatically reduces output token count — which is where most API cost lives.
Trigger-based, not poll-based	Agents only run when something happens: a user queries a new subscription (Layer 1), a check-in is recorded (Layer 2), or an impulse flag is raised (Layer 3). This keeps costs minimal and interventions timely.